

Шпаргалка: общие приёмы решения алгоритмических задач.

Файл пополняется по мере разбора задач. Каждый приём — с примером из проекта.

1. Как придумать условие цикла.

Условие цикла — это ответ на вопрос «когда работа ещё НЕ закончена?». Идёшь от обратного: сначала формулируешь, когда можно остановиться и результат уже корректен, а потом берёшь отрицание.

Порядок действий:

1. Сформулируй, что является «работой» цикла (например: перенести все элементы на свои места).
2. Возьми самое честное условие — «пока не вся работа сделана» (например: `writeIndex >= 0`, «пока есть куда писать»). Оно почти всегда корректно, и на собеседовании его достаточно.
3. Потом спроси себя: «есть ли ситуация, когда работа фактически закончена раньше?». Если да — сузь условие. Если нет — оставь как есть, это не ошибка.

Важно: часто корректны сразу несколько условий, «единственно правильного» нет.

Отдельный вопрос — строгое сравнение или нестрогое. Он решается тем, включена ли граница в ещё не сделанную работу. Если позиция, на которую указывает граница, ещё не проверена, условие обязано быть нестрогим, иначе последний элемент останется непросмотренным.

Пример — `BinarySearch` (LeetCode 704): границы `lowIndex` и `highIndex` включительные, поэтому при `lowIndex == highIndex` на участке ещё остаётся один непроверенный элемент, и условие цикла — `lowIndex <= highIndex`. Со строгим `<` на массиве из

одного элемента цикл не начался бы вовсе, и метод вернул бы «не найдено» при правильном ответе 0.

Пример — `MergeSortedArray` (LeetCode 88): верны все три варианта — `writeIndex >= 0`, `firstIndex >= 0 || secondIndex >= 0` и минимальный `secondIndex >= 0` (остатки `nums1` уже стоят на своих местах, а остатки `nums2` надо докопировать).

2. Два неравноправных источника: смотри на донора.

Когда в задаче два массива (или два указателя) неравноправны — один является приёмником (в него пишем результат), а другой донором (из него переливаем) — условие завершения цикла обычно привязано к донору: его надо исчерпать полностью. Приёмник «доливать сам в себя» не нужно.

Пример — `MergeSortedArray`: донор — `nums2`, поэтому минимальное условие цикла — `while (secondIndex >= 0)`.

Пример — `MoveZeroes` (LeetCode 283): донор — читающий указатель, он обязан просмотреть каждый элемент, поэтому условие первого цикла привязано к нему (`readIndex < nums.length`), а не к пишущему. Пишущий указатель отстаёт ровно на число пропущенных нулей, и его финальная позиция — уже результат работы, а не условие остановки: с неё начинается второй цикл, досыпающий нули.

3. «Для всех» vs «существует»: где стоит return.

Где размещать `return` относительно цикла, определяется типом утверждения, которое проверяет алгоритм.

Утверждение «для ВСЕХ элементов верно P» (например, «все пары символов совпадают»): одно совпадение ничего не доказывает, а одно несовпадение опровергает всё сразу. Поэтому опровержение (`return false`) — внутри цикла, срабатывает мгновенно; подтверждение (`return true`) — после цикла, когда прошли всё и не нашли ни одного контрпримера.

Утверждение «СУЩЕСТВУЕТ элемент со свойством P» (например, «есть пара с нужной суммой»): зеркально. Одна находка доказывает существование — подтверждение (`return` находки) внутри цикла; опровержение («не найдено») — после цикла, когда перебрали всё впустую.

Правило-памятка: `return` после цикла — это фолбэк, самая частая пропущенная ветка в тестах (см. соглашения в CLAUDE.md: у каждого `return` — свой тест).

Примеры: `ValidPalindrome` (LeetCode 125) — «для всех»: `return false` внутри цикла при несовпадении пары, `return true` после цикла; `TwoSumSorted` (LeetCode 167) — «существует»: `return` найденной пары внутри цикла, `return {-1, -1}` после; `ValidParentheses` (LeetCode 20) — тоже «для всех»: два `return false` внутри цикла (пустой стек и несовпадение типа), подтверждение после цикла. Но подтверждение здесь не голое `true` — см. приём 4; `ValidAnagram` (LeetCode 242) — «для всех» в чистом виде, но во ВТОРОМ цикле: утверждение «все 26 счётчиков равны нулю», поэтому `return false` при первой же ненулевой ячейке, а `return true` — после того как прошли весь массив; `FirstUniqueChar` (LeetCode 387) — «существует»: `return currentIndex` при первом же символе со счётчиком 1, а `return -1` — после цикла, когда просмотрена вся строка и уникальных не нашлось; `ContainsDuplicate` (LeetCode 217) — «существует» в чистом виде: `return true` при первом же повторе, `return false` — после цикла. Здесь особенно соблазнительно поставить `return false` внутрь, в ветку «этого значения ещё не было», — но одно уникальное значение ничего не доказывает, надо дойти до конца; `BinarySearch` (LeetCode 704) — «существует»: `return middleIndex` при совпадении внутри цикла, `return -1` — после. Отличие от предыдущих в том, что цикл здесь не перебирает элементы подряд, а сужает участок; но тип утверждения от этого не меняется, и фолбэк стоит там же.; `SymmetricTree` (LeetCode 101) — «для всех» без цикла, в рекурсии: утверждение «все пары узлов, стоящих напротив друг друга, совпадают». Роль `return false` внутри цикла играет `&&`: первое ложное условие обрывает цепочку, и в оставшиеся пары рекурсия не заходит; `true` получается только когда проверены все пары до самых листьев.

Пример от обратного — `SearchInsertPosition` (LeetCode 35): тот же цикл, но утверждения «существует» в задаче нет вовсе. Ответ есть всегда: либо индекс найденного значения, либо позиция для вставки. Поэтому после цикла стоит не фолбэк, а полноценный ответ `return lowIndex`, и отдельного теста «на случай, когда ничего не нашли», здесь не бывает — этот случай такой же обычный, как и находка. Полезно проверять себя этим вопросом: «а есть ли у задачи исход *ответа нет?*». Если нет, то и фолбэка быть не должно.

4. Копишь состояние — проверь, что оно израсходовано.

Продолжение приёма 3. Если алгоритм по ходу цикла накапливает состояние (стек, счётчик, остаток, буфер), то «прошли всё и не нашли нарушений» ещё не означает «ответ да». Надо отдельно убедиться, что накопитель пуст — то есть вся отложенная работа доделана.

Признак: внутри цикла есть шаг «отложить на потом» (`push`, `+1`, «запомнить»), а нарушение ловится только в момент разбора отложенного. Тогда элементы, отложенные и НЕ разобранные, никакой ветки внутри цикла не задевают — цикл про них просто молчит.

Правило-памятка: финальный `return` после цикла — не `true`, а проверка накопителя (`stack.isEmpty()`, `counter == 0`, `balance == 0`).

Пример — `ValidParentheses` (LeetCode 20): строка "(" не нарушает ни одного правила внутри цикла, потому что закрывающих скобок в ней нет вообще. Ошибка видна только по остатку в стеке, поэтому в конце стоит `return openBrackets.isEmpty()`, а не `return true`.

Пример — `ValidAnagram` (LeetCode 242): цикл только копит счётчики букв (`+1` за букву из `s`, `-1` за букву из `t`) и ничего не сравнивает, поэтому нарушений внутри него не бывает в принципе. Ответ целиком определяется остатком: массив счётчиков обязан обнулиться. Здесь накопитель ещё нагляднее, чем стек, — «израсходован» буквально означает «все счётчики равны нулю».

Тот же приём в вырожденном варианте с одним типом скобок: там стек заменяется счётчиком, и финальная проверка — `balance == 0`.

5. Правишь массив на месте — следи, чтобы писатель не обгонял читателя.

Когда результат пишется в тот же массив, откуда читаются данные, единственная настоящая опасность — затереть значение, которое ещё не прочитано. Значит перед тем как писать, надо сформулировать инвариант: В МОМЕНТ ЗАПИСИ пишущий указатель позади читающего (или на той же позиции). Если инвариант держится, решение in-place корректно; если нет — нужно менять направление обхода или заводить копию.

Оговорка про момент записи не придирка: между записями указатели могут на мгновение поменяться местами (после `writeIndex++` он равен `readIndex + 1`, пока не увеличился сам `readIndex`). Проверять инвариант надо там, где происходит присваивание, иначе легко «доказать» себе несуществующую ошибку.

Порядок действий:

1. Определи, кто читает, а кто пишет.
2. Проверь, может ли пишущий указатель оказаться впереди читающего хоть на одном шаге. Достаточно посмотреть, кто из них сдвигается чаще.
3. Если может — попробуй развернуть обход: часто «то же самое, но с конца» чинит задачу без дополнительной памяти.

Пример — `MoveZeroes` (LeetCode 283): `writeIndex` сдвигается только на ненулевых элементах, а `readIndex` — на каждом, поэтому `writeIndex <= readIndex` выполняется всегда. Худший случай — оба на одной позиции, и элемент копируется сам в себя.

Пример — `MergeSortedArray` (LeetCode 88): при слиянии с начала пишущий указатель сразу обгонял бы читающего по `nums1` и затирал бы значимые элементы. Спасает разворот направления: пишем с конца, в хвост из заглушек, и позиция записи всегда правее обоих читающих указателей.

6. Ответ зависит от того, что впереди — нужен предварительный проход.

Перед тем как писать цикл, спроси: чтобы принять решение про элемент на позиции `i`, достаточно ли того, что уже просмотрено слева? Если решение зависит от элементов ПРАВЕЕ `i`, одним проходом задача не решается в принципе — на момент прохода этой информации ещё нет.

Лечится это предварительным проходом: первый цикл собирает сводку по всему входу (частоты, суммы, максимумы), второй по этой сводке ищет ответ. Два прохода сложность не портят: $O(n) + O(n) = O(n)$.

Отдельно проверь, ПО ЧЕМУ идёт второй цикл. Сводка обычно перетасовывает порядок: массив счётчиков хранит буквы по алфавиту, а не по позиции в строке. Если в ответе важен порядок исходных элементов, второй цикл обязан идти по исходным данным, а сводка использоваться только как справочник.

Пример — `FirstUniqueChar` (LeetCode 387): уникальность первого символа зависит от того, встретится ли он ещё раз дальше, поэтому сначала считаем частоты, потом ищем. Второй цикл идёт по СТРОКЕ: обход по массиву счётчиков вернул бы алфавитно первую уникальную букву вместо первой по позиции. На `"ccdaab"` это дало бы `b` (индекс 5) вместо правильного `d` (индекс 2).

Пример — `ValidAnagram` (LeetCode 242): та же схема, но проще, потому что порядок в ответе не важен вовсе. Поэтому второй цикл спокойно идёт по массиву счётчиков.

7. Придумал ключ-отпечаток — проверь его на склейку.

Часть задач сводится к тому, чтобы построить для элемента «отпечаток»: величину, одинаковую у всех элементов одной группы и разную у элементов из разных групп. Дальше группировка или сравнение делаются по нему за $O(1)$.

«Отпечаток» — рабочее название, устоявшегося русского термина нет; подробнее в `glossary.md`.

Отпечаток обязан удовлетворять двум условиям сразу, и второе забывают:

1. у эквивалентных элементов отпечатки СОВПАДАЮТ;
2. у неэквивалентных — РАЗЛИЧАЮТСЯ.

Первое обычно очевидно из построения. Второе — нет: отпечаток часто склеивает разные элементы в один ключ, и группы молча сливаются. Ошибка не даёт ни исключения, ни падения теста на маленьком примере.

Порядок действий: построил ключ — сразу придумай два РАЗНЫХ элемента, которые могли бы дать одинаковый ключ. Не получилось придумать — хорошо, ключ однозначен. Получилось — нужен разделитель, длина или другое поле, разводящее эти случаи.

Пример — `GroupAnagrams` (LeetCode 49): ключ из вектора частот выписывается через разделитель, `1#0#11#...`. Без разделителя слово с одной `a` и одиннадцатью `b` даёт `111`, и слово с одиннадцатью `a` и одной `b` — тоже `111`. Две разные группы схлопнулись бы в одну. На тестах вида `["eat","tea"]` это не всплывает вовсе: чтобы поймать склейку, счётчик должен перевалить за десять.

Тот же приём на нестроковых данных: координаты точки, склеенные как `"12"`, — это и `(1, 2)`, и `(12, ...)`. Разделитель нужен ровно по той же причине.

8. Восстанови инвариант до того, как измеришь результат.

Когда алгоритм ведёт объект с инвариантом — окно без повторов, отрезок с суммой не больше `X`, стек в согласованном состоянии, — внутри итерации есть три разных действия, и путать их порядок нельзя:

1. починить нарушение (сжать, откатить, выбросить лишнее);
2. применить изменение (добавить новый элемент);
3. измерить результат (обновить максимум, записать ответ).

Измерение идёт последним. Меряя раньше починки, ты снимаешь показание с объекта, который в этот момент инвариант нарушает, — то есть с состояния, которое ответом быть не может в принципе.

Порядок действий:

1. Сформулируй инвариант одной фразой: «в окне нет повторов», «сумма не больше `x`». Если сформулировать не получается, инварианта нет и приём не про эту задачу.
2. Определи, какое действие может его нарушить. Обычно это ровно одно место — добавление нового элемента.
3. Поставь починку сразу за нарушением, а измерение — после починки.

Ошибка почти всегда даёт не падение, а завышенный ответ: измерили объект в тот момент, когда он был больше допустимого.

Пример — `LongestSubstringNoRepeat` (LeetCode 3): инвариант «в окне нет повторов». Нарушить его может только входящий символ, поэтому цикл сжатия стоит перед добавлением символа, а `Math.max` — после. На строке `"abba"` измерение до сжатия дало бы на позиции 2 длину 3 (`"abb"`, с повтором внутри) вместо правильного ответа 2.

Пример от обратного — `MaxAverageSubarray` (LeetCode 643): у окна фиксированной ширины инвариант «ширина ровно `k`» нарушается и чинится одной строкой, `+ вошедший - минус вышедший`, поэтому отдельного цикла починки нет. Приём замечен именно там, где починка занимает переменное число шагов.

9. Каждая итерация обязана уменьшать задачу.

Цикл заканчивается не потому, что так задумано, а потому, что величина из условия цикла двигается в нужную сторону на КАЖДОЙ итерации. Если есть хоть один путь через тело, на котором ничего не изменилось, цикл вечный.

Проверять надо именно так: пройти по всем веткам тела и на каждой спросить — что уменьшилось? Ветка без ответа на этот вопрос и есть зависание. У цикла со счётчиком (`for (int i = 0; i < n; i++)`) вопрос снят самой формой записи, поэтому приём нужен там, где границы двигаются условно — внутри `if`.

Отдельная ловушка — когда сужение записано «до проверенного элемента включительно»: `lowIndex = middleIndex` вместо `middleIndex + 1`.

Формально участок вроде бы сжимается, но при длине участка два элемента середина округляется влево и совпадает с левой границей, то есть не меняется ничего.

Пример — `BinarySearch` (LeetCode 704): на массиве `[1, 2]` и `target = 2` середина равна 0, значение мало, и присваивание `lowIndex = middleIndex` оставило бы `lowIndex = 0` навсегда. Единица (`middleIndex + 1`) не поправка на всякий случай, а точная запись того, что про средний элемент уже известно: он сравнён и не подошёл, значит выбрасывается вместе с половиной.

Пример — `LongestSubstringNoRepeat` (LeetCode 3): внутренний `while` сжимает окно слева, и завершается он потому, что каждая итерация двигает левую границу вперёд на единицу. Если бы граница двигалась только при выполнении дополнительного условия, цикл мог бы не закончиться.

Пример — `MaxDepthBinaryTree` (LeetCode 104) и `SymmetricTree` (LeetCode 101): то же правило в рекурсивном виде. Каждый вызов спускается на уровень ниже — по одному узлу в первой задаче, сразу по двум во второй, — а дном служит пустое поддерево. Рекурсия без базового случая — то же, что цикл без движения границ.

10. Цикл меняет то, по чему идёт, — зафиксируй границу до входа.

Если тело цикла добавляет элементы в ту же структуру, по которой цикл идёт, условие вида `i < collection.size()` перечитывается на каждой итерации и «уезжает»: цикл обрабатывает и то, что сам же добавил. Когда это не нужно, границу снимают ДО цикла в отдельную переменную и идут до неё.

Признак: внутри цикла есть `add` / `push` / `offer` в ту же коллекцию, размер которой стоит в условии.

Правило-памятка: `int count = queue.size(); for (i < count)`, а не `for (i < queue.size())`.

Пример — `LevelOrderTraversal` (LeetCode 102): внутренний цикл снимает узлы уровня с головы очереди и кладёт их потомков в хвост.

`nodesOnLevel` снимается в начале итерации внешнего цикла, пока очередь содержит ровно один уровень; с условием `taken < nodesToVisit.size()` цикл прошёл бы всё дерево одним уровнем.